

# Shiv Nadar University Chennai

## CS3807 – Deep Learning Laboratory

### Experiment 2

## Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27 |

## 1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

## 2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

## 3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size:  $28 \times 28$

## 4. Image Preprocessing

Fashion-MNIST images are stored as  $28 \times 28$  matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to  $[0,1]$ .
5. Convert labels into one-hot vectors.

## 5. Source Code

**GitHub Repo Link:** <https://github.com/likithainnamuri7/Deep-learning>

**Google Colab Notebook:** <https://colab.research.google.com/drive/1c7Kpu5G0jqZGEJjE7pswzFJQiTqH--G0usp=sharing>

## 6. Experimental Procedure

### Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

**Expected Output:** Dataset summary and sample images.

### Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

### Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

### Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

### Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

## 7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

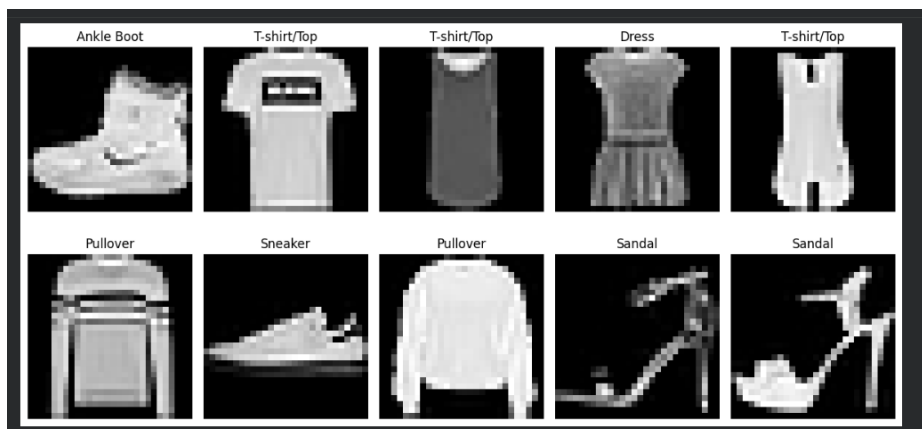
| Hyperparameter          | Candidate Values    |
|-------------------------|---------------------|
| Hidden Layers           | 1, 2, 3             |
| Hidden Neurons          | 32, 64, 128, 256    |
| Learning Rate           | 0.1, 0.01, 0.001    |
| Batch Size              | 16, 32, 64, 128     |
| Epochs                  | 10, 20, 30          |
| Optimizer               | SGD, Adam, RMSProp  |
| Activation Function     | ReLU, Tanh, Sigmoid |
| Dropout Rate (Optional) | 0.0, 0.2, 0.5       |

### Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

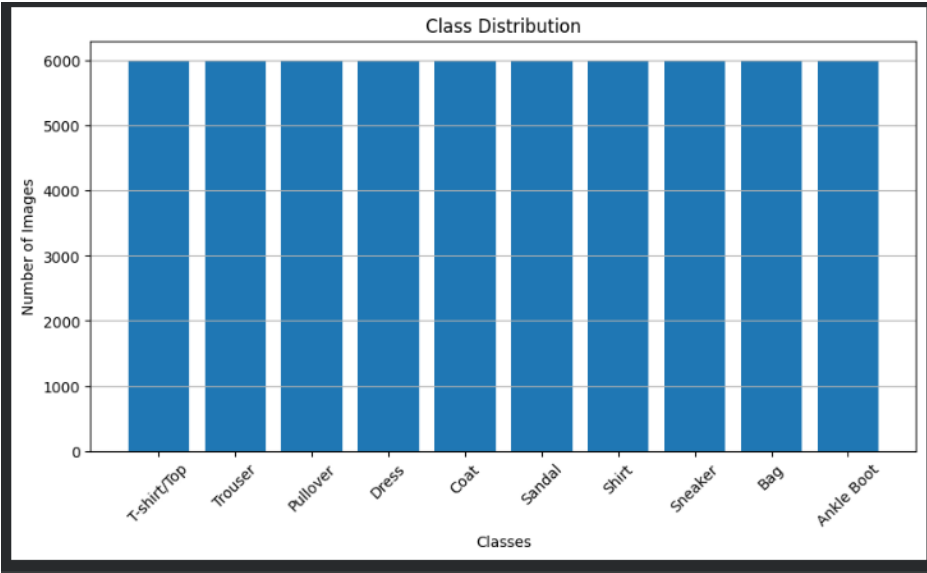
## 8. Mandatory Plots

### 8.1 Sample Images



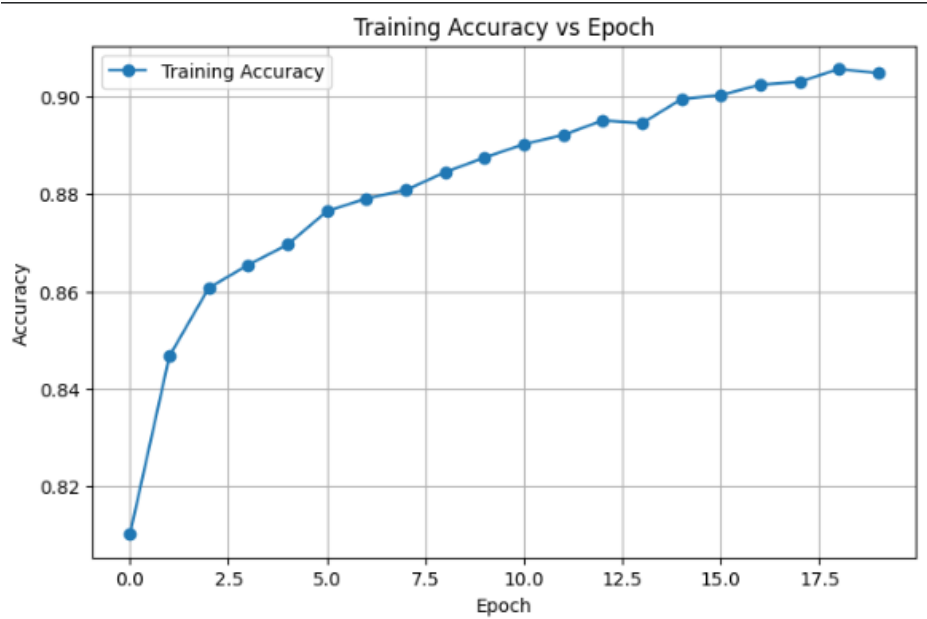
**Inference:** These sample images shows the different categories of clothing items in grayscale with a resolution of 28x28 pixels. The dataset consists of a variety of classes, making it suitable for evaluation of the multi-class classification capability of MLP model.

## 8.2 Class Distribution



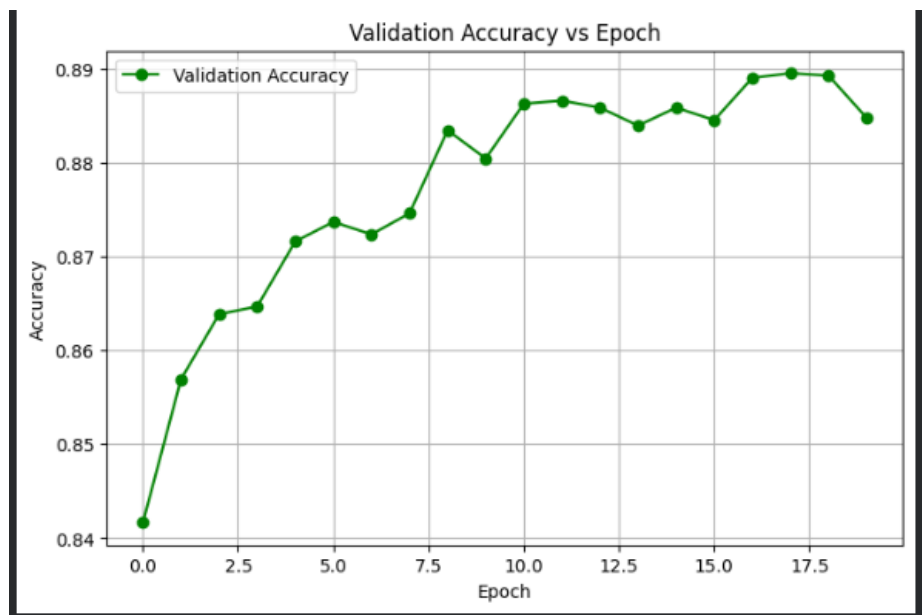
**Inference:** The class distribution is perfectly balanced,each class has 6,000 training images.since there is no class imbalance,the model can equally learn from all the actegories without bias towards any particular class.

## 8.3 Training Accuracy vs Epoch



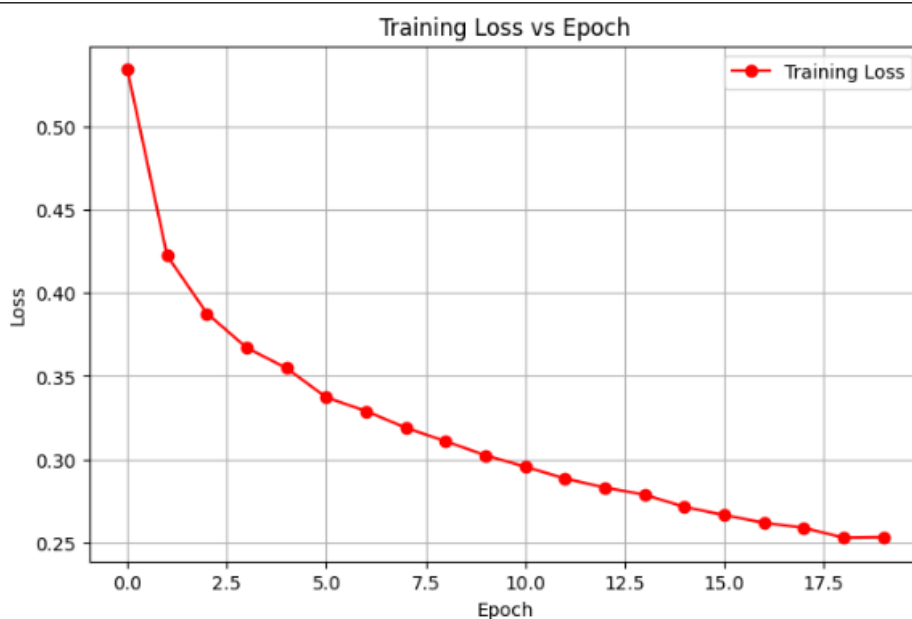
**Inference:**The training accuracy is increasing steadily from approximately 81that the model is learning effectively,learning the underlying patterns in the training data.The gradually improvement suggests stable convergence without any sudden fluctuations.

## 8.4 Validation Accuracy vs Epoch



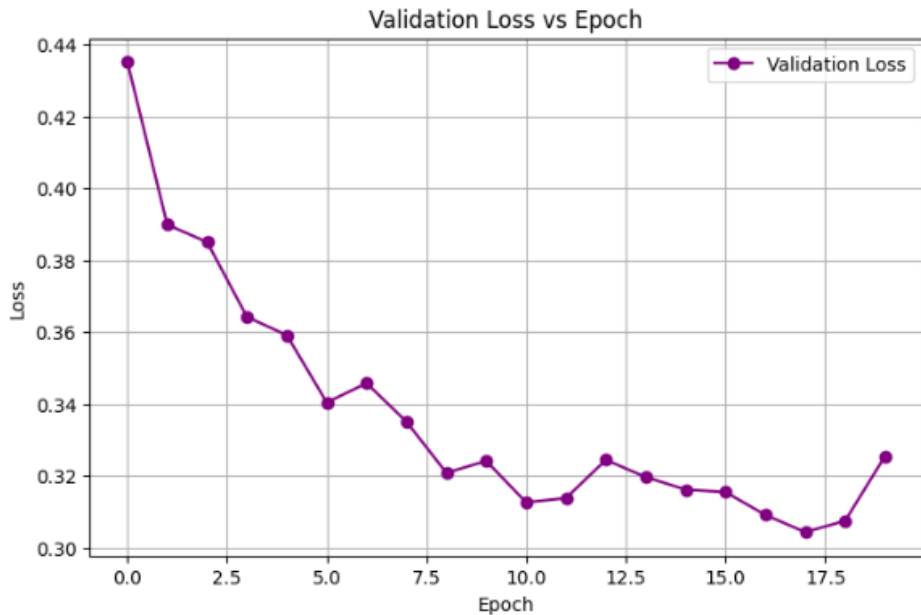
**Inference:** The validation accuracy is improved from 84 between the training accuracy and the validation accuracy indicates that the model is generalising well with signs model generalizes well to unseen data without significant overfitting.

## 8.5 Training Loss vs Epoch



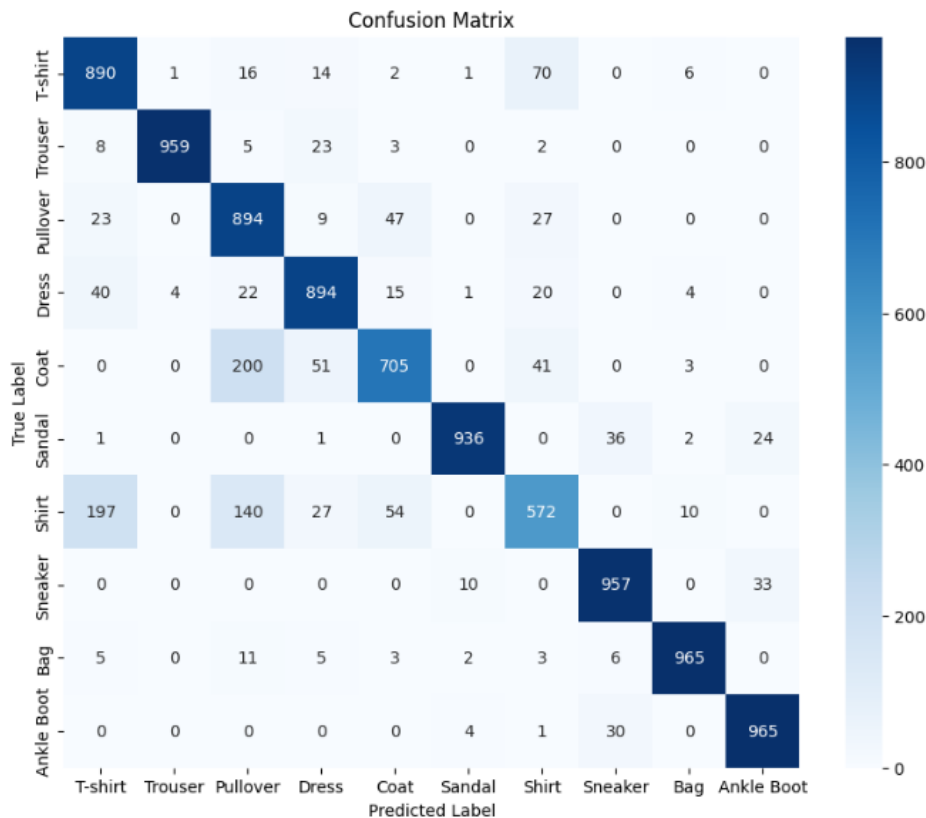
**Inference:** The training loss decreases consistently from 0.54 to 0.25 shows that prediction error reduces as training progresses. This indicates that the optimiser successfully minimizing the loss function and improving the models performance.

8.6 Validation Loss vs Epoch



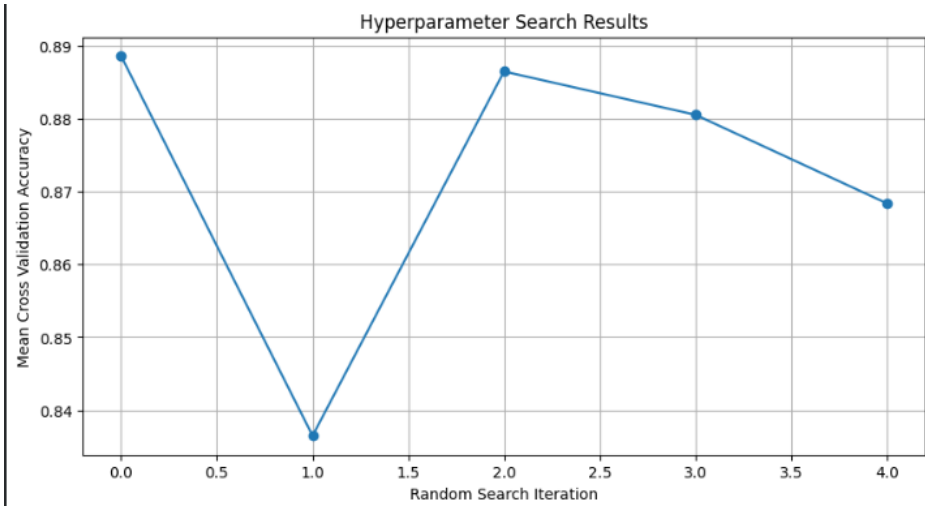
**Inference:** The validation loss decreases during the intial epochs and reaches its minimum around at epochs 17-18 after which it slightly increases.This tells that mild overfitting in the later stages of training process.

8.7 Confusion Matrix



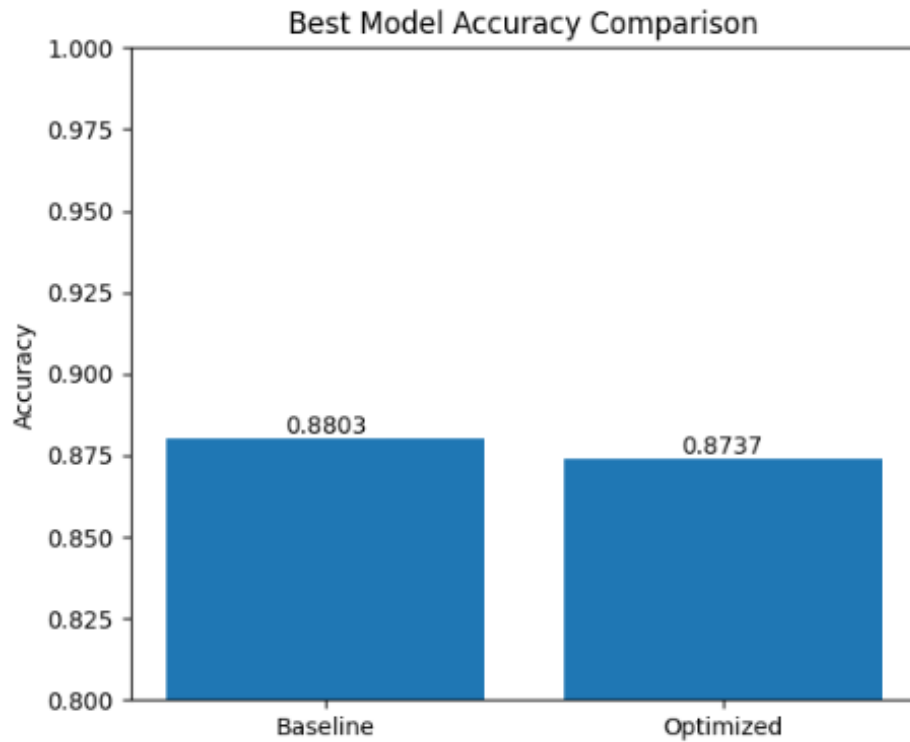
**Inference:** The confusion matrix shows that most classes are classified correctly, particularly Trouser, Sandal, Sneaker, Bag, and Ankle Boot, which are those having high diagonal values. Misclassifications mainly occur among visually similar clothing categories such as Shirt, Pullover, Coat, and T-shirt/Top

8.8 Hyperparameter Search Results



**Inference:**The RandomizedSearchCV is evaluated multiple hyperparameter combinations, and the cross-validation accuracies varied across the different trials. The best combination achieved the highest validation accuracy, demonstrating the effectiveness of automated hyperparameter tuning in exploring different model configurations.

## 8.9 Best Model Accuracy Comparison



**Inference:** Hyperparameter optimization using RandomizedSearchCV does not guarantee better performance than the baseline model. Since it evaluates only randomly selected hyperparameter combinations, the chosen configuration may not generalize better on the test dataset. increasing the number of search iterations or exploring additional combinations may improve the optimized model's performance.

## 9. Results

### Best Hyperparameters

|                           |        |
|---------------------------|--------|
| Hidden Layers             | 1      |
| Hidden Neurons            | 256    |
| Learning Rate             | 0.001  |
| Batch Size                | 64     |
| Optimizer                 | Adam   |
| Activation Function       | Tanh   |
| Epochs                    | 20     |
| Dropout                   | 0.2    |
| Cross-validation Accuracy | 88.86% |
| Testing Accuracy          | 87.37% |

## Performance Comparison

| Metric    | Baseline | Optimized |
|-----------|----------|-----------|
| Accuracy  | 88.03%   | 87.37%    |
| Precision | 88.04%   | 87.40%    |
| Recall    | 88.03%   | 87.37%    |
| F1-score  | 88.02%   | 87.36%    |

## 10. Discussion

### 1 Hyperparameter Optimization Analysis

#### 1.1 Which optimization method was used?

RandomizedSearchCV with 5-fold cross-validation was used to optimize the MLP model. Instead of testing every possible combination, it randomly selects different hyperparameter combinations, making the search more efficient while still finding a good performing model.

#### 1.2 Which hyperparameters were selected?

The best model used the following hyperparameters:

- Number of hidden layers: 1
- Hidden neurons: 256
- Optimizer: Adam
- Activation function: Tanh
- Learning rate: 0.001
- Batch size: 64
- Number of epochs: 20
- Dropout rate: 0.2

These settings achieved the highest crossvalidation accuracy among the tested sampled combinations.

#### 1.3 Did optimization improve performance?

No. although the optimized model achieved the best crossvalidation performance during hyperparameter tuning, its testing accuracy (87.37%) was slightly lower than the baseline model (88.03%). This shows that hyperparameter optimization does not always guarantee better performance on unseen test data.

#### 1.4 Which hyperparameter had the greatest impact?

The number of hidden neurons and the learning rate had the greatest impact on the model's performance. These parameters directly affected the model's learning capacity and convergence, leading to noticeable changes in accuracy.

## 1.5 Compare Grid Search and Randomized Search

| Method            | Advantages                                                                                                                                         | Disadvantages                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Grid Search       | Evaluates every possible hyperparameter combination and usually finds the optimal solution.                                                        | Requires significantly more computation time, especially for large search spaces.      |
| Randomized Search | Tests only a random subset of combinations, making it faster and more practical for large search spaces while still achieving competitive results. | May not test the absolute best combination because not all possibilities are explored. |

## 1.6 Recommended MLP Configuration

For this dataset, an MLP with the following configuration is recommended:

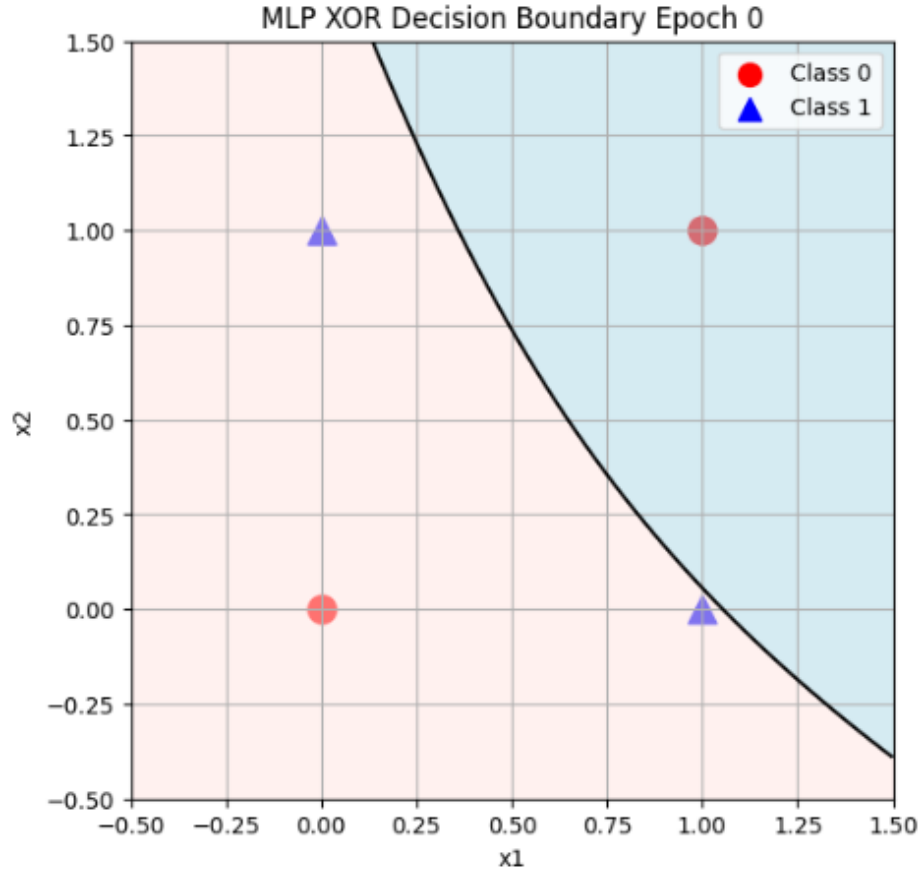
- Hidden layers: 1
- Hidden neurons: 256
- Activation function: Tanh
- Optimizer: Adam
- Learning rate: 0.001
- Batch size: 64
- Epochs: 20
- Dropout: 0.2

This configuration achieved the best cross-validation performance and provided good generalization on the Fashion-MNIST dataset.

## 2 Perceptron Learning Algorithm for XOR Gate

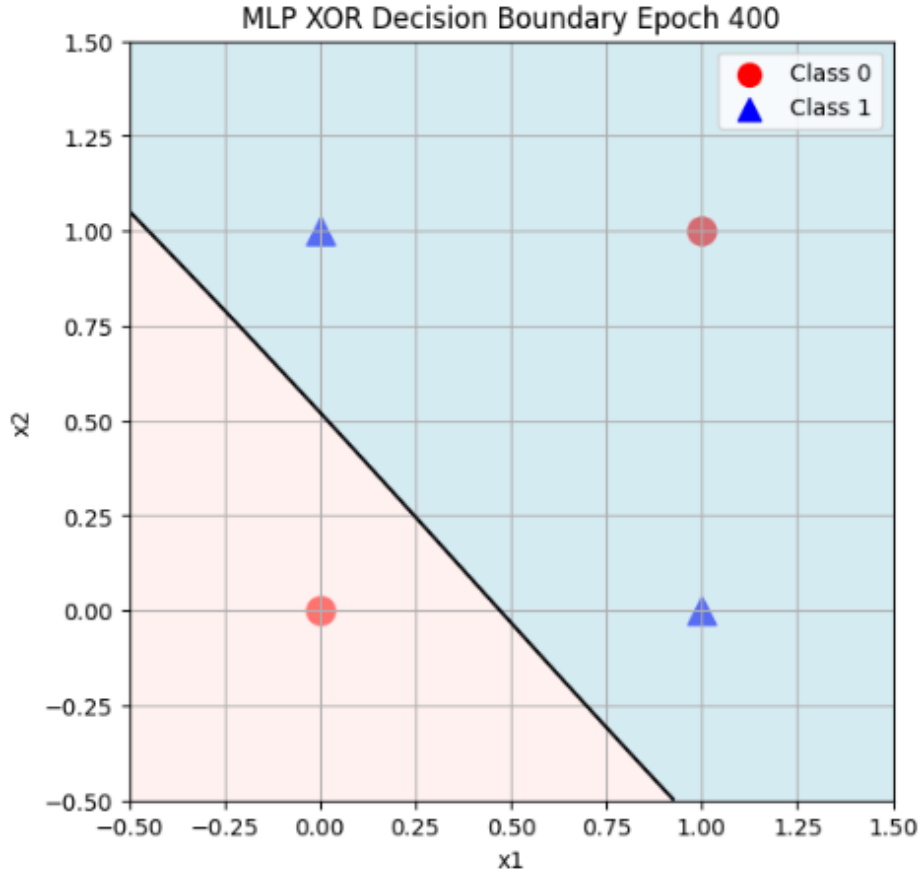
The XOR gate was implemented using a Multi-Layer Perceptron (MLP) with one hidden layer trained using the backpropagation algorithm. The decision boundary was plotted at different training epochs to visualize how the network gradually learns the nonlinear XOR relationship. The weights and biases were updated after every iteration until convergence.

## 2.1 Epoch 0



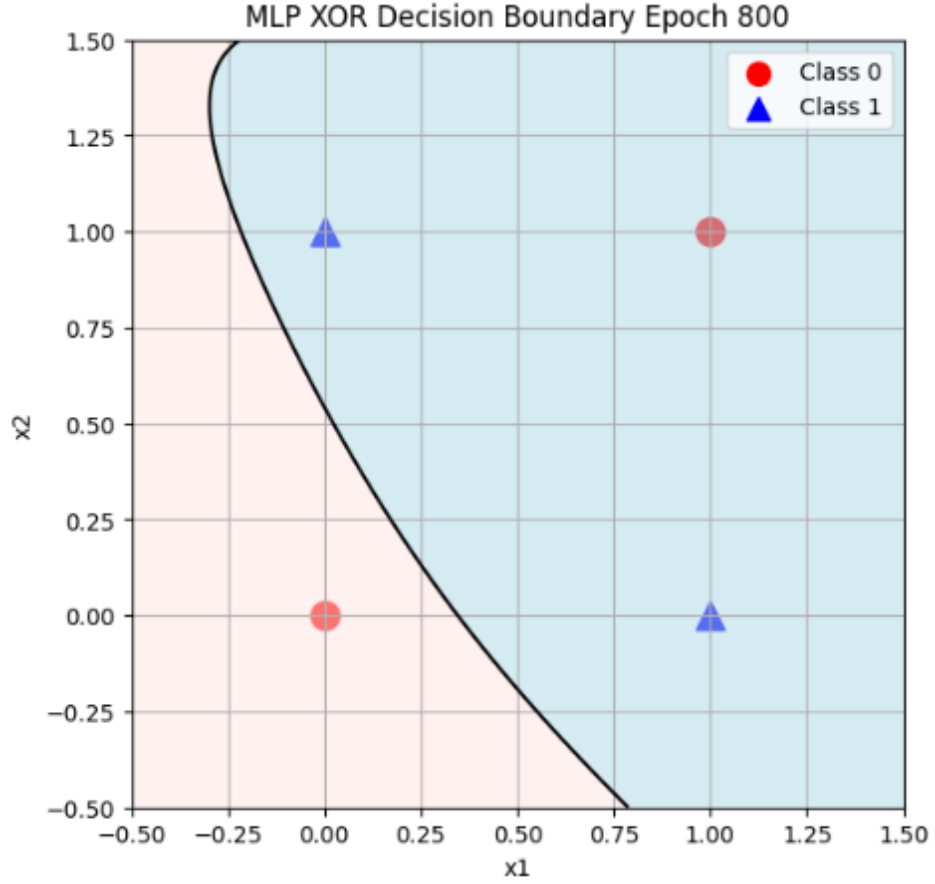
**Inference:** At the beginning of training, the network starts with randomly initialized weights. The hidden-layer weights are  $W_1 = \begin{bmatrix} 1.6207 & -0.6246 \\ -0.5255 & -1.0821 \end{bmatrix}$ , while the output-layer weights are  $W_2 = \begin{bmatrix} 0.8750 \\ -2.2899 \end{bmatrix}$ . The hidden bias values are very close to zero (0.0026, -0.0225) and the output bias is 0.0285, indicating that no meaningful learning has occurred yet. Consequently, the decision boundary is almost random and fails to separate the XOR classes, as XOR is inherently a nonlinear problem.

## 2.2 Epoch 400



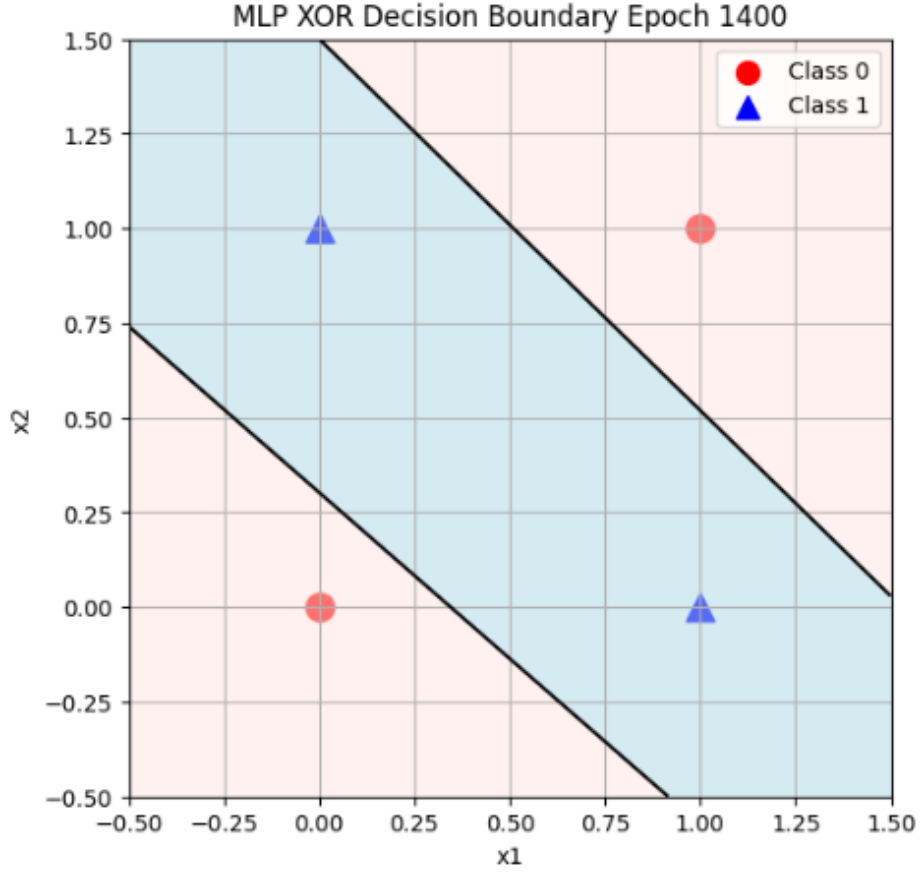
**Inference:** After 400 epochs, the network has started learning useful feature representations. The hidden-layer weights have increased significantly to approximately  $W_1 = \begin{bmatrix} 0.5982 & -3.8441 \\ -1.4039 & -3.9750 \end{bmatrix}$ , while the output weights become  $W_2 = \begin{bmatrix} 0.6858 \\ -4.2036 \end{bmatrix}$ . The larger negative weights indicate that the hidden neurons are becoming more selective toward different input combinations. The decision boundary begins to curve, showing the first signs of nonlinear separation, although all XOR points are not yet perfectly classified.

### 2.3 Epoch 800



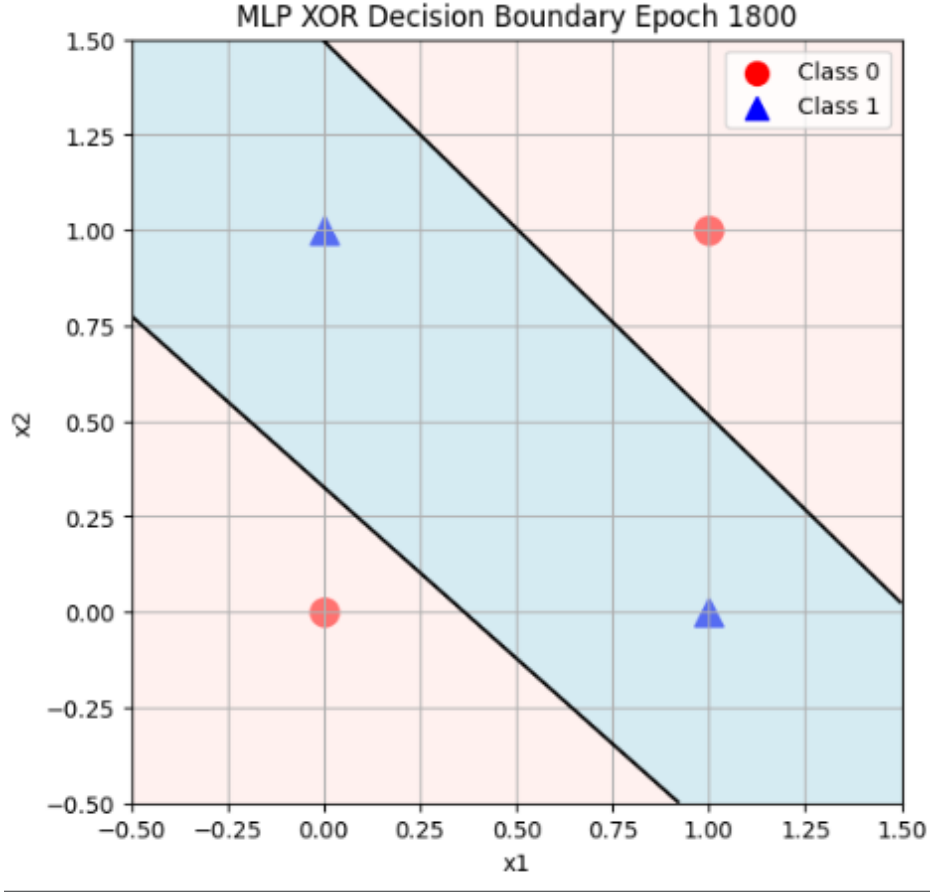
**Inference:** By epoch 800, the learning process has progressed considerably. The hiddenlayer weights now reach values close to  $-5.47$ , while the output-layer weights become  $W_2 = \begin{bmatrix} 2.9115 \\ -5.3976 \end{bmatrix}$ . The hidden biases have also increased to  $(0.3910, 0.8700)$ , indicating stronger neuron activation. The decision boundary becomes highly nonlinear and bends around the data points. At this stage, most of the XOR samples are correctly classified, demonstrating that the hidden layer is successfully transforming the original feature space into one where the classes are easier to separate.

## 2.4 Epoch 1400



**Inference:** At epoch 1400, the network is very close to convergence. The hiddenlayer weights stabilize around  $W_1 = \begin{bmatrix} -3.2012 & -5.5315 \\ -3.2676 & -6.2685 \end{bmatrix}$ , while the output weights increase further to  $W_2 = \begin{bmatrix} 6.4749 \\ -7.1681 \end{bmatrix}$ . The hidden biases have grown to (4.6102, 1.8742) and the output bias becomes  $-2.7652$ . The decision boundary clearly forms distinct nonlinear regions that successfully separate the two XOR classes. Compared with earlier epochs, only small refinements in the boundary are observed, indicating that the model has almost completed learning.

## 2.5 Epoch 1800



**Inference:** At epoch 1800, the network has effectively converged. The hiddenlayer weights become  $W_1 = \begin{bmatrix} -3.5971 & -5.7346 \\ -3.6650 & -6.3687 \end{bmatrix}$ , and the output weights reach  $W_2 = \begin{bmatrix} 7.2960 \\ -7.7343 \end{bmatrix}$ . compared to epoch 1400, the weight changes are very small, confirming that the optimization process has stabilized. The decision boundary completely separates the XOR classes, demonstrating that the MLP has successfully learned the nonlinear XOR mapping through hidden-layer feature extraction.

## 2.6 Final Prediction

| Input | Probability | Predicted Class |
|-------|-------------|-----------------|
| (0,0) | 0.0469      | 0               |
| (0,1) | 0.9452      | 1               |
| (1,0) | 0.9438      | 1               |
| (1,1) | 0.0737      | 0               |

**Inference:** The trained network correctly predicts all four XOR input combinations with high confidence. The positiveclass inputs (0,1) and (1,0) obtain probabilities of 94.52% and 94.38%, respectively, while the negativeclass inputs (0,0) and (1,1) receive very low probabilities of only 4.69% and 7.37%. These results confirm that the model has fully converged and successfully learned the XOR function by creating a nonlinear decision boundary that cannot be achieved using a single-layer perceptron.

## 2.7 Analysis

The XOR problem is not linearly separable, meaning that no single straight line can correctly divide the two output classes. A singlelayer perceptron would continuously update its weights without converging because it can only learn linear decision boundaries. In contrast, the implemented Multilayer Perceptron introduces a hidden layer with sigmoid activation, enabling nonlinear feature transformation. As training progresses from epoch 0 to epoch 1800, the weight magnitudes increase steadily, the hidden neurons learn increasingly discriminative features, and the decision boundary evolves from an arbitrary linear like separator into a complex nonlinear boundary. This transformation allows the network to correctly classify all XOR patterns with prediction probabilities greater than 94% for the positive class and less than 8% for the negative class, demonstrating successful convergence of the MLP.

## 11. Conclusion

In this experiment, a MultiLayer Perceptron was successfully implemented for both Fashion-MNIST image classification and the XOR gate problem. For the Fashion-MNIST dataset, the model achieved good classification performance after preprocessing, training, and evaluation, while hyperparameter optimization using RandomizedSearchCV identified an effective model configuration, although it did not outperform the baseline model on the test dataset. For the XOR problem, the MLP successfully learned the nonlinear decision boundary through backpropagation, correctly classifying all four input combinations with prediction probabilities above 94% for the positive class and below 8% for the negative class. The experiment demonstrated the importance of hidden layers in solving nonlinearly separable problems and provided practical understanding of MLP architecture, backpropagation, model evaluation, decision boundary visualization, and hyperparameter optimization in deep learning.

## 12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.